

RabbitMQ 활용 현황 점검

작성일: 2026-08-17
대상: "크롤링 요청 큐가 RabbitMQ의 어떤 기능을 실제로 쓰고 있는지, 어떤 건 코드만 있고 안 쓰는지"를 확인하려는 개발자
전제: [크롤러 수집 요청 큐](#), [백엔드 관리자 대시보드 전체 동작 프로세스](#)가
전체 파이프라인을 다룬다면, 이 문서는 그중 RabbitMQ 하나만 떼어서 "필요했는가"와 "기능을 얼마나 쓰는가"를 감사(audit)한다.
이 문서 작성 시점 기준으로 ①재시도 시 우선순위가 리셋되던 버그와 ②idempotencyKey가 계산만 되고 중복 차단에 쓰이지 않던 문제는 같은 세션에서 코드로 수정하고 Docker 통합 테스트까지 통과시켰다. 아래 표는 그 수정 이후 기준이다.

1. RabbitMQ가 필요했는가

이 시스템이 풀어야 했던 요구사항 네 가지가 메시지 브로커를 정당화한다.

요구사항	근거
이기종 워커 경쟁 소비	Python Worker와 Go Worker가 같은 큐(<code>purchase-research.collection.search.v1</code>)를 동시에 구독할 수 있어야 함(Python→Go 전환 검증 기간의 설계 전제)
우선순위 기반 처리 순서	배경 사전 갱신(5) < 기본 요청(20) < 반응형 갱신(30) < 구매 전 재검증(100) — 브로커가 큐 안에서 순서를 재정렬해줘야 함
지연 재시도(retry-with-backoff)	판매처 응답 실패를 곧바로 재시도하지 않고 5초 뒤 재투입해야 함
HTTP 스레드의 비동기 분리	크롤링 한 건이 최대 30초 걸릴 수 있는데, 관리자가 폼을 제출한 순간 HTTP 응답은 broker ACK(최대 5초)만 기다리면 됨

다만 짚어야 할 점: `collection_tasks / collection_jobs` 는 발행 전에 이미 PostgreSQL에 `QUEUED` 로 저장된다(`CollectionTaskPublisher.publish()` 가 `collectionJobService.register(...)` 를 `publishTask(...)` 보다 먼저 호출). 즉 이 시스템의 진짜 상태 원천은 이미 Postgres이고, RabbitMQ는 그 위에 얹힌 **배달·실행순서 메커니즘**이다. 나쁜 설계는 아니지만(견고한 outbox 패턴에 가깝다), Redis도 이미 docker-compose에 떠 있으면서 대부분 놓고 있다는 점에서 "RabbitMQ가 불필요했다"보다는 "인프라 구성이 최소는 아니다"가 더 정확한 평가다.

2. 구현되어 실제로 동작하는 것

기능	근거(file:line)
durable exchange/queue + persistent 메시지	<code>rabbitmq.py:265-302</code> 전부 <code>durable=True</code> , <code>delivery_mode=aio_pika.DeliveryMode.PERSISTENT</code> ; <code>RabbitCollectionConfiguration.java</code> 전 Bean <code>QueueBuilder.durable(...)</code>
Publisher confirm (broker ACK까지 대기)	<code>CollectionTaskPublisher.java:179-187</code> <code>correlationData.getFuture().get(5, SECONDS)</code> ; <code>rabbitmq.py:146</code> <code>channel(publisher_confirms=True)</code>
mandatory + return 처리 (라우팅 실패 감지)	Java <code>application.yaml:22-24</code> <code>publisher-returns: true</code> , <code>template.mandatory: true</code> ; Python <code>rabbitmq.py:146</code> <code>on_return_raises=True</code> , <code>_publish()</code> 가 <code>mandatory=True</code>

기능	근거(file:line)
우선순위 큐(<code>x-max-priority</code>)	<code>x-max-priority: 100</code> 선언(<code>rabbitmq.py:281</code> , <code>RabbitCollectionConfiguration.java:52</code>), 발행 시 실제 AMQP 속성에 <code>setPriority()</code> 로 심음(<code>CollectionTaskPublisher.java:397</code>)
재시도 메시지도 우선순위 유지	이번 세션에 수정: <code>_publish()</code> 가 <code>priority</code> 파라미터를 받아 재시도 발행에도 원본 우선순위를 그대로 적용(<code>rabbitmq.py</code>) — 이전에는 재시도 시 우선순위가 소실됐음
TTL+DLX 지연 재시도	별도 플러그인 없이 <code>x-message-ttl: 5000</code> + <code>dead-letter</code> 로 원래 큐 복귀(<code>rabbitmq.py:284-292</code> , <code>RabbitCollectionConfiguration.java:61-68</code>)
Dead Letter Exchange/Queue	검색.결과 큐 모두 DLX 설정, <code>search.dlq.v1</code> / <code>result.dlq.v1</code> 로 격리
Manual ack/nack + prefetch	Python <code>prefetch_count=1</code> (<code>rabbitmq.py:147</code>); Java <code>application.yaml:26-31</code> <code>acknowledge-mode: manual</code> , <code>concurrency: 1</code> , <code>prefetch: 1</code>
자동 재연결(connection recovery)	Python <code>aiopika.connect_robust</code> (<code>rabbitmq.py:144</code>); Spring AMQP 기본 <code>recovery</code>
idempotencyKey 기반 애플리케이션 중복 발행 차단	이번 세션에 구현: <code>collection_tasks.idempotency_key</code> (V13 마이그레이션) + <code>existsByIdempotencyKeyAndStatusIn(...)</code> 으로 QUEUED/RUNNING 중복만 차단, 종료된 과거 작업은 막지 않음
토폴로지 선언 일치 (Python↔Java)	<code>exchange/queue</code> 이름, <code>durable</code> , <code>x-dead-letter-*</code> , <code>x-message-ttl</code> , <code>x-max-priority</code> 가 완전히 동일 — 어느 서비스가 먼저 떠도 406 PRECONDITION_FAILED 충돌 없음

3. 코드단에만 남아있음(구현은 있으나 기본은 비활성/미사용)

기능	상태	근거
경쟁 소비자(competing consumer)로 수평 확장	코드.설정은 지원하지만 운영상 워커 1개만 실행	Python/Go 워커를 여러 프로세스로 띄우면 브로커가 자동 분배하는 구조지만, "Python→Go 전환 검증 중에는 하나만 실행"이 현재 운영 정책
Go Collector Worker의 실제 큐 소비	코드는 같은 큐(<code>purchase-research.collection.search.v1</code>)를 구독할 수 있게 돼 있지만 현재 기본 운영에서는 Python Worker만 켜짐	시스템 구조 문서: "두 runtime은 같은 검색 Queue를 경쟁 소비하므로 전환 검증 중에는 하나만 실행"
<code>CollectionFreshnessScheduler</code> (신선도 기반 사전 재수집)	완전히 구현됐지만 기본값이 꺼짐	<code>application.yaml:57-58</code> <code>purchase-research.freshness.scheduler.enabled: false</code> 가 기본값, <code>@ConditionalOnProperty(... , havingValue = "true")</code>
Redis 기반 <code>MerchantRateLimiter</code>	완전히 구현됐지만 옵트인	<code>create_rate_limiter_from_env()</code> 는 <code>PURCHASE_RESEARCH_REDIS_URL</code> 환경변수가 있을 때만 활성화 (<code>rate_limiter.py:103-119</code>). 꺼진 채로 워커를 여러 개 띄우면 판매처별 전역 <code>rate limit</code> 이 보장되지 않음
<code>DetailFreshnessCache</code> (상세조회 신선도 캐시)	RabbitMQ 큐 경로에서는 아예 참조하지 않음	파일 상단 주석에 명시: Queue 작업은 이미 상품 50개·30초로 작아서 이 캐시가 필요 없고,

기능	상태	근거
		<code>scripts/run_unified_crawl.py</code> 같은 수동 대량 실행 스크립트 전용

4. 참고 — 애초에 시스템에 없는 기능(필요 없거나 PoC 단계라 보류)

기능	코드에 아예 없음	이유
Quorum/Mirrored 큐(HA)	✓	<code>compose.yaml:44-60</code> 이 단일 노드 <code>rabbitmq:4.2</code> , 두 topology 선언 모두 <code>x-queue-type</code> 인자 없음(classic 단일 노드 큐) → 이 노드가 죽으면 즉시 가용성 단절. PoC 단계라 감안 가능하나 프로덕션 전환 시 반드시 짚어야 함
Delayed-message 플러그인	✓	TTL+DLX 트릭으로 이미 대체했으므로 플러그인 자체가 불필요
RPC(<code>reply-to / correlation-id</code>) 패턴	✓	상태 조회가 폴링 기반 Agent Run으로 설계돼 있어("짧고 상한 있는 polling", 종료되지 않는 polling thread는 안 만듦) 애초에 필요 없는 패턴
Topic/Header exchange, 와일드카드 라우팅	✓	routing key가 고정된 1:1 매핑이라 지금의 <code>direct</code> exchange가 정확한 선택. 굳이 topic/header를 썼다면 불필요한 복잡도
브로커 레벨 메시지 중복 제거 플러그인	✓	RabbitMQ 기본 기능이 아니므로, 대신 §2의 idempotencyKey를 앱(DB) 레벨에서 검사하는 방식으로 구현 — 이 선택이 맞음

5. 결론

"신뢰성 있는 작업 큐"에 필요한 핵심 기능(영속성, publisher confirm, mandatory/return, 우선순위, TTL 지연 재시도, DLQ, manual ack + prefetch, 자동 재연결)은 전부 쓰고 있고 올바르게 구현돼 있다. 반대로 §3의 "코드단에만 남아있는" 항목들은 대부분 **PoC/전환 검증 단계라 의도적으로 꺼둔 것이고**, §4의 항목들은 **애초에 이 시스템 요구사항에 안 맞아서 안 만든 것이다**. 두 그룹을 혼동하지 않는 게 중요하다 — §3은 "설정 한 줄 켜면 바로 쓸 수 있는 완성된 기능"이고, §4는 "필요해지면 새로 설계해야 하는 기능"이다. 프로덕션 전환을 논의한다면 우선순위는 §3(특히 멀티 워커 스케일아웃, Redis rate limiter 활성화)이 높고, HA 큐(§4)는 그다음이다.